

Technická dokumentace – Sail Connect

1. Použitý framework a struktura projektu

Aplikace **Sail Connect** je implementována jako **Single Page Application (SPA)** s využitím knihovny **React** a jazyka **TypeScript**. Projekt je vytvořen pomocí nástroje **Vite**, který zajišťuje rychlý vývojový server a moderní build pipeline.

Hlavní použité technologie

- **React 18** – komponentový UI framework
- **TypeScript** – statická typová kontrola
- **Vite** – build nástroj a dev server
- **React Router** – klientské routování
- **React Hook Form + Zod** – správa a validace formulářů
- **Vitest** – unit testy
- **LocalStorage** – persistence dat (bez backendu)

Struktura projektu (zjednodušeně)

src/

```
├-- features/
|   ├── auth/      (auth logika, context, repo, typy)
|   ├── trips/     (plavby – komponenty, repo, utils, schema)
|   ├── bookings/  (rezervace – repo, schema)
|
├-- pages/         (stránky aplikace – routy)
├-- components/    (sdílené UI komponenty)
├-- layout/        (MainLayout – header/footer)
├-- test/          (unit testy)
├-- lib/           (pomocné utility – storage)
└-- router/        (definice rout)
```

Aplikace je strukturována **feature-first** přístupem – každá doménová oblast (auth, trips, bookings) má vlastní logiku, typy, validaci i testy.

2. Architektura komponent

Aplikace je rozdělena na:

Stránkové komponenty (Pages)

Stránky odpovídají jednotlivým routám aplikace:

- HomePage – seznam plaveb + filtry
- TripDetailPage – detail plavby + rezervace
- DashboardPage – přehled uživatele
- CreateOfferPage, EditOfferPage
- BookingDetailPage, EditBookingPage
- UserDetailPage, EditUserPage

Tyto komponenty:

- pracují s routami,
- skládají menší komponenty,
- řeší navigaci a autorizaci.

Prezentační komponenty

Např.:

- TripCard
- TripList
- TripFilters

Jsou:

- znovupoužitelné,
- bez znalosti persistence,
- řízené přes props.

Layout

MainLayout obsahuje:

- Header
- Footer

Layout je sdílený pro celou aplikaci.

3. Správa stavu a dat

Stav aplikace

Použity jsou tři úrovně správy stavu:

Lokální stav komponent

- useState, useMemo
- např. filtry plaveb, stav formuláře, UI stav

Kontext – AuthContext

Pro globální stav přihlášeného uživatele:

- aktuální uživatel (user)
- metody login, logout, register
- reaktivní aktualizace hlavičky a chráněných rout

AuthContext je jediný globální context v aplikaci a slouží výhradně pro autentizaci.

Repo vrstva + LocalStorage

Persistence dat je řešena přes **repo pattern**:

- auth/repo.ts
- trips/repo.ts
- bookings/repo.ts

Repo:

- abstrahuje přístup k LocalStorage,
- poskytuje CRUD operace,
- odděluje data od UI.

Používané klíče v LocalStorage:

- users, session
- userTrips
- bookings

4. Validace a formuláře

Formuláře jsou řešeny pomocí:

- **React Hook Form** – výkon a jednoduchost
- **Zod** – centrální a typově bezpečná validace

Každý formulář má:

- vlastní Zod schema (createOfferSchema, bookingSchema, editUserSchema),
- automatickou validaci,
- chybové hlášky zobrazované u polí.

Validace probíhá:

- při blur,
- při submitu,
- jednotně napříč aplikací.

5. Testování

Použité nástroje

- **Vitest**
- **jsdom** prostředí
- testy ve složce src/test

Typy testů

- **unit testy utilit** (např. filtrace plaveb)
- **testy repo vrstev** (CRUD operace)
- **testy validačních schémat (Zod)**

Příklady:

- validace správného a chybného vstupu formuláře,
- přidání / úprava / smazání rezervace,
- aktualizace nabídky.

Testy lze spustit:

npm run test

npm run test:run

6. Optimalizace a přístupnost

Výkon

- lazy loading obrázků (loading="lazy")
- responsive obrázky (srcSet, sizes)
- WebP formát
- memoizace výpočtů (useMemo)

Přístupnost (Accessibility)

- správné <label> pro formuláře
- sémantické HTML
- textové alternativy obrázků (alt)
- dostatečný kontrast barev
- klávesová ovladatelnost

7. Shrnutí

Aplikace Sail Connect je navržena jako moderní klientská SPA aplikace s důrazem na:

- přehlednou architekturu,
- oddělení logiky od UI,
- typovou bezpečnost,
- testovatelnost,
- výkon a přístupnost.